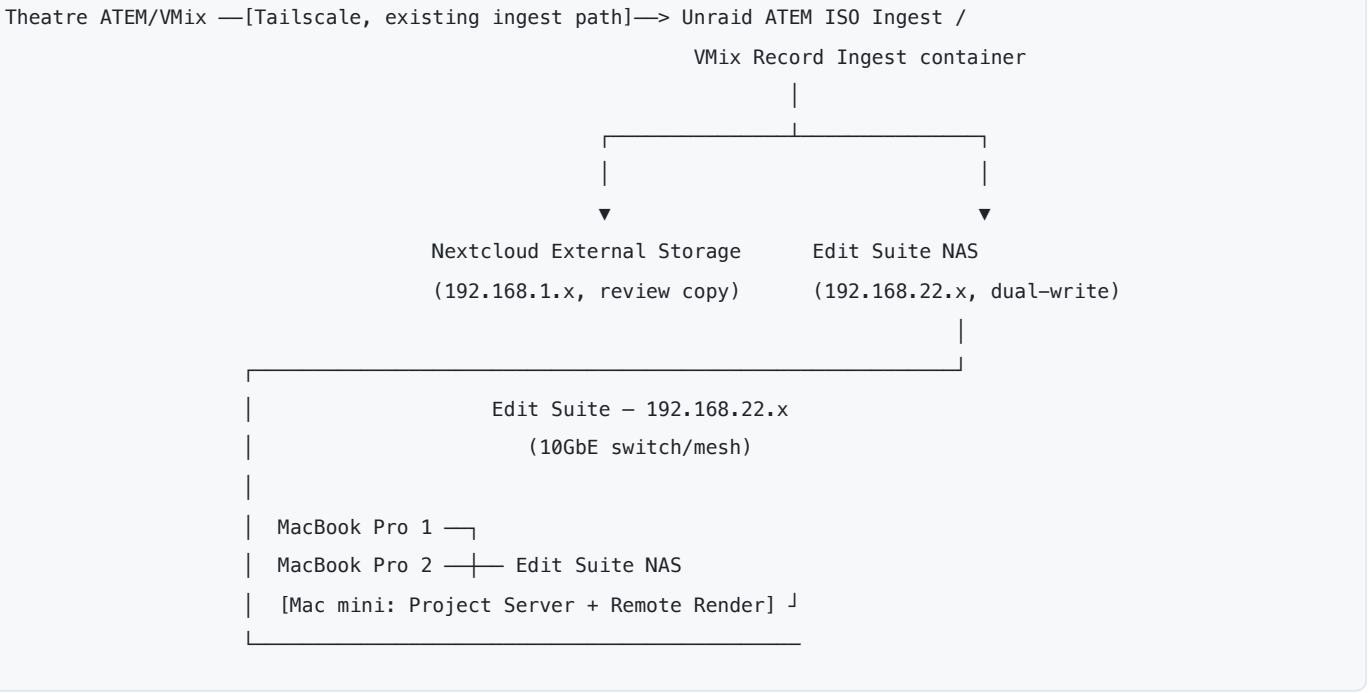
Live editing — post-production at the mothership

A separate subsystem at the mothership: 2× MacBook Pro editing workstations plus dedicated fast storage, editing event content as it arrives rather than waiting until after the event. Deliberately **not** part of the theatre-facing network this whole repo otherwise documents — different traffic pattern (sustained multi-gigabit scrubbing/export, not the bursty low-bitrate streams everything else here is sized for), different security boundary (no reason for 12 theatres to reach an editor's workstation or vice versa), and a genuinely different failure domain (an editing workstation crashing shouldn't touch the live show).

Why this needs its own network segment, not just a corner of the existing one

Every bandwidth figure elsewhere in this repo (`docs/bandwidth-analysis.md`) is sized around what the *live event* needs — SRT/NDI streaming, ATEM ISO ingest, file sync. Editing is a different animal: scrubbing or exporting 4K footage wants sustained throughput in the hundreds of Mbps to multiple Gbps *per editor*, not the tens of Mbps this design budgets per theatre. Putting that load on the mothership's existing 2× bonded 1GbE (`docs/server-specification.md`) would mean either starving the editors or risking contention with the live ingest pipelines still running during the event — the one thing this whole design goes out of its way to avoid elsewhere (see the NDI-fallback mitigation's own margin concerns in `docs/bandwidth-analysis.md`).

Recommendation: a dedicated 10GbE LAN for the edit suite, physically separate from the mothership's theatre-facing network, proposed as `192.168.22.0/24` (next free block after the VMix nodes — see `docs/ip-address-map.md`):



Same ingest containers already documented in `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md`, writing to a **second destination** as well as Nextcloud — one pull off the ATEM/VMix source, two writes, not a chained re-sync (see Decision 1 below). No Tailscale within the edit suite itself — everything in this subsystem is in the same room as the mothership, unlike the theatre uplinks this repo's Tailscale mesh exists to solve for (`docs/tailscale.md`). A simple routed connection back to `192.168.1.x` is enough for anything that still needs it; the edit suite doesn't need to be reachable *from* any theatre, and vice versa.

Standard MacBook Pros don't have built-in 10GbE — each needs a Thunderbolt-to-10GbE adapter or dock (e.g. OWC/Sonnet-class hardware) to actually hit these speeds; budget for that alongside whatever storage is chosen below.

See `diagrams/live-editing-dataflow.svg` for this same dual-write path as a diagram.

Decision 1: Nextcloud's storage vs. a dedicated NAS

Dedicated NAS, kept in sync via the same rclone/rsync pattern already built for ingest. Three reasons Nextcloud's own storage (the Unraid recording pool) is the wrong target to edit directly against:

- **It's sized and provisioned for the ingest workload, not editing.** The recording pool's whole design brief (`docs/server-specification.md`) is absorbing 16 concurrent low-bitrate append streams reliably — a completely different I/O shape from 2 editors scrubbing/exporting simultaneously.
- **Contention risk during the live event.** ATEM ISO Ingest and VMix Record Ingest are still actively writing to this same pool while editors would be reading from it — exactly the kind of shared-resource risk this repo's segmentation philosophy exists to avoid everywhere else.
- **Nextcloud's own serving stack (PHP/WebDAV) adds overhead a raw SMB/NFS-native NAS mount doesn't have** — fine for occasional file sync, not ideal for sustained media scrubbing.

A dedicated NAS, fed by the **same pull, writing to a second destination** — not a chained re-sync reading Nextcloud's own copy back out again, just the existing `config/atem-iso-ingest/` and `config/vmix-record-ingest/` containers' `rsync --append` step writing to both `/mnt/user/nextcloud-external/...` (Nextcloud's External Storage, for the near-real-time review path this repo already documents) and a second mount on the edit-suite NAS, in the same pass. One read off the ATEM/VMix source, two writes — cheaper and simpler than pulling twice, and the edit suite never has to wait on Nextcloud's own indexing (`occ files:scan`) to see new footage, since it's not going through Nextcloud at all. Same incremental-transfer reasoning as everywhere else in this repo: `rsync --append` only transfers new bytes as footage keeps growing, on both destinations. (The second write is a documented design, not yet a code change — the actual scripts currently write only the Nextcloud destination; tracked as `docs/open-questions.md` item 15.)

NAS options, roughly by cost:

Option	~Price	Notes
Blackmagic Cloud Dock 4 / Dock 2 / Pod (BYO drives)	$445-$1,535	Pure 10GbE network enclosure, no included storage — cheapest path to real throughput if drives are sourced separately
Generic 10GbE NAS (Synology/QNAP-class, populate with own drives)	Varies, comparable to Cloud Dock + drives	Not Resolve-specific, but plain SMB/NFS works identically; more flexible (can also run Docker containers — relevant to Decision 2)

Option	~Price	Notes
Blackmagic Cloud Store Mini, 8TB	**$4,945**	4× M.2 NVMe, **RAID 0 — no redundancy**, 1×10GbE + 1×1GbE, up to 50 concurrent connections. Confirm the RAID0 risk is acceptable — a drive failure mid-event means falling back to a fresh pull from the mothership's own recording pool, not losing anything permanently, but it is a real mid-event disruption
Blackmagic Cloud Store Mini, 16TB	$8,245	Same, more headroom if editing against more than a curated subset of the event's footage

For 2 editors working from a curated subset of footage (not the full 8TB recording pool — see the open question below on how much actually needs to be pulled), the Cloud Store Mini 8TB is a reasonable fit and gets Resolve-aware sync to Blackmagic Cloud "for free" if that's ever wanted later (see Decision 4). A generic 10GbE NAS is the cost-conscious or redundancy-conscious alternative. Either way, the Project Server is a separate dedicated Mac mini (Decision 2), not something hosted on the NAS itself — see below.

Decision 2: a dedicated Mac mini running the Project Server and Remote Render

The workflow this section is built around (see below) has **resolve-configurator build one shared show project** — a single set of Theatre/Date bins and per-session timelines both editors work against, populated by smart bins as footage lands — not two editors each on their own private copy. That design choice settles the "do we even need Collaboration" question from an earlier draft of this doc: **yes**, because both editors need to see the same live-updating bin/timeline structure, not separate forks of it. So this is a dedicated node, not an optional one.

What that node needs to run:

- **The DaVinci Resolve Project Server** — a small standalone app bundling a pinned PostgreSQL version, syncing project metadata (bins, timelines, locks) between editors in real time. **It does not move media** — actual footage still needs to be reachable by both editors via shared storage regardless (the NAS from Decision 1).
- **Resolve's Remote Render feature** — dispatches an export job from an editor's Deliver page to a separate networked Resolve Studio install, so the editor's own laptop isn't tied up rendering while cutting continues. Confirmed (Blackmagic's own Reference Manual): this is one job to one machine at a time — **not** a distributed render farm; Resolve never splits a single job's frames across multiple nodes.

Recommendation: a single dedicated Mac mini running both, rather than a Docker container on a NAS (dropped from an earlier draft — Blackmagic ships the Project Server as a native macOS/Windows/Linux app, not a container image, so this isn't a realistic option) and rather than either editor's own laptop. The most-cited failure mode across DIT/post-production forum threads is exactly the laptop case: if that machine sleeps, reboots, or the lid closes, *both* editors lose the shared project, not just one.

Combining Project Server and Remote Render duty on one Mac mini — confirmed workable, but thinly documented, so flagging the confidence honestly:

- Blackmagic's own documentation does not state a prohibition on co-locating the two roles, and architecturally there's no reason it wouldn't work — the Project Server is just a background

PostgreSQL-backed service, Remote Render is just a licensed Resolve Studio instance running in a render role. But this specific combination isn't something Blackmagic explicitly confirms in writing, and it's a thin pattern in practitioner discussion — one forum thread asked this exact question ("could the project library be on this Mac mini as well?") and never got a direct answer; one real-world example found keeps a small Mac mini as project-server-only, separate from any render node. Worth treating as **very likely fine for a 2-editor event-scale deployment, not a Blackmagic-certified pattern** — a reasonable design decision, not a guaranteed one.

- **Both the render node and every editor's machine need DaVinci Resolve Studio**, not the free version — confirmed directly in Blackmagic's manual ("remote rendering does not work with the free version of DaVinci Resolve"). **$295 perpetual license per seat**, one-time cost. Blackmagic's own forum states an activation-code license can be active on 2 machines simultaneously, which in principle could cover an editor's laptop plus the render node from one seat — but this is a general Studio licensing term applied to this scenario, not a Remote-Render-specific clause, so budget one license per machine unless that's tested and confirmed to work as expected.
- **Remote Render requires a Network project library (PostgreSQL), not a Local one** — confirmed by a Blackmagic DaVinci support engineer on their own forum. In other words, Remote Render can't be added on top of two independent local projects; it only works once the Project Server (or Blackmagic Cloud) infrastructure already exists. That's further reason these two decisions are coupled here, not separable.
- **The media storage volume must be mounted on both the requesting machine and the render node** — confirmed in Blackmagic's manual. The Mac mini needs the same Edit Suite NAS mount the MacBook Pros use, not just database connectivity.
- **Performance is workable for 1080p H.264 but not fast** — no source benchmarks this exact scenario, so treat this as reasoned inference from adjacent data: Apple Silicon's H.264 hardware encode gap (base/Pro chips: one encode engine vs. two on Max, four on Ultra) matters far more at 4K/UHD than at 1080p, where independent reviews found little real difference between a Mac mini and a Mac Studio. But remote rendering itself runs slower than local rendering regardless of hardware — one documented case measured roughly half the fps remotely vs. locally. **Spec the Mac mini at M2 Pro/M4 Pro tier or better**, not the base chip, given it's doing double duty as database host and render node; a base-tier mini is a real risk for anything beyond light 1080p H.264 jobs.
- **Known gotcha to plan around**: submitting multiple render jobs to the same remote node at the same time can silently fail — send them one at a time. Fine at 2-editor scale, worth documenting as an operational note rather than something to engineer around.

Decision 3: using Blackmagic Cloud Store as the project server

No — not currently supported, and not just "not recommended." Blackmagic Cloud Store appliances are documented as SMB/NFS media storage plus a Resolve-aware sync target for the separate Blackmagic Cloud service (Decision 4) — nothing in Blackmagic's own documentation or independent DIT/post-production sources indicates they expose a general-purpose compute or container environment capable of hosting the Project Server's PostgreSQL database. Practitioner guidance consistently treats "where the media lives" (Cloud Store or any NAS) and "where the project database lives" (the dedicated Mac mini from Decision 2) as two separate questions with two separate answers — the project server needs its own host regardless of which storage option was picked.

Decision 4: the cloud store's built-in internet sync functionality

Not needed for this design, and probably not worth turning on. "Blackmagic Cloud" is a genuinely separate product from the physical Cloud Store hardware — an internet-based sync service (own subscription: **~$5/month per project library** + **~$15/TB/month** for synced media) built for the case where collaborators are in *different physical locations* and don't want to deal with VPNs/firewalls to reach each other. Everything in this design is in one room at one venue — there's no second site to sync with, so this service would add an ongoing subscription cost and an internet-facing dependency for a problem this topology doesn't have. **Worth reconsidering only if there's a genuine need for someone off-site (a remote producer, a head-office stakeholder) to review cuts** — that's a real use case Blackmagic Cloud is built for, just a different one from what's being solved here. Track as a possible future need, not a current requirement.

Workflow: from ATEM pull to finished edit

Everything above (dedicated NAS, project-server decision) exists to support one actual pipeline — footage should already be sitting where an editor expects it, organized by session, by the time anyone sits down to cut. The pieces:

```
Theatre ATEM/VMix ──[existing ingest containers]──> dual write:
                                                    ├─> Nextcloud (review copy)
                                                    └─> Edit Suite NAS
                                                              │
                                                              ▼
                        resolve-configurator (run ahead of the event, from the same
                        session CSV nc-filedropbatch already uses for presenter
                        uploads) has already built, into a shell Resolve project:
                          • one bin per theatre, per day
                          • one empty timeline per session inside each day's bin
                          • backgrounds/title-slide assets, pre-imported
                          • a smart-bin RECIPE per session (Resolve's scripting API
                            can't create real smart bins — applied once, by hand,
                            before the event; see resolve-configurator's own README
                            for why)
                                                              │
                                                              ▼
                           (one-time, pre-event: apply each recipe in Resolve's UI
                            to turn it into a real smart bin)
                                                              ▼
                    As footage lands on the NAS, each session's smart bin auto-fills
                    with just its own clips (filtered by record-drive path + date +
                    the session's timecode window) — no manual sorting.
                                                              │
                                                              ▼
                         Editor opens that session's pre-built timeline, drags the
                         now-populated smart bin's clips in, top-and-tails, exports
                         (optionally via Remote Render — see Decision 2).
```

What resolve-configurator actually builds (from its own README, not assumed): project format settings (frame rate/resolution, or any Resolve project setting via passthrough), a `Theatre / Date` bin structure, an empty timeline per session named from a template (e.g. `Start Time – Presenter`), pre-imported background/title-slide assets (global or per-theatre), and — since Resolve's scripting API genuinely cannot create smart bins, confirmed for the versions currently in use — a written recipe (`smart-bins.md` / `smart-bins.json`) giving the exact filter rule for each session (file path contains that theatre's record-drive name, date created is the session date, start timecode falls in the session's derived window). The tool builds everything the API *can* do automatically; the smart bins themselves are a one-time manual step per session, applied once ahead of the event from that recipe — after which they behave exactly like any other Resolve smart bin, auto-filtering as matching footage arrives.

Once that one-time setup is done, the editor's own job is genuinely just: open the session's timeline (already sitting in the right day/theatre bin, already named), the smart bin next to it already shows only that session's clips (no scrubbing through a whole day's unsorted recordings), drag them onto the timeline, top-and-tail to the actual session length, export. All of the organizing work happens ahead of time or automatically — editors are cutting, not filing.

See `diagrams/live-editing-project-structure.svg` for this whole pipeline as a diagram — from the session CSV through resolve-configurator's build, the smart-bin recipe's one-time hand-apply step, and the auto-fill down to the editor's own four-step job.

Ingest workflow: physical media (master drives, camera cards)

A separate, complementary path to the network-based ATEM ISO pull already built (`docs/atem-iso-ingest.md`) — that pipeline is optimized for near-real-time *review* access, but the ATEM's own USB SSD (and any standalone camera's SD/CFexpress cards, if this event uses cameras beyond the NDI-fed BirdDog P400s) remains the authoritative master, same "master vs. mirror" framing as the network ingest doc. Physically walking a drive/card to the edit suite for a proper checksummed offload — not just relying on the network pull — is standard DIT (Digital Imaging Technician) practice for anything that matters enough to edit.

Tool comparison

Tool	Vendor	Price	Role
ShotPut Pro	Imagine Products	$169 perpetual (+$59-70/yr updates after year 1), $60/30-day rental	Industry-standard dedicated checksummed offload + verify + PDF/CSV/MHL report — the classic "DIT cart" tool
OffShoot / OffShoot Pro	Hedge	$149-249 one-time, $49/30-day rental	Same core job (fast checksummed offload, multiple simultaneous destinations), simpler UI, no metadata/look-management layer
Silverstack Offload Manager	Pomfort	$139/yr, project licenses from $35	Pomfort's own cut-down offload-only tier, positioned against ShotPut/OffShoot for smaller productions
Silverstack XT / Lab	Pomfort	~$899/yr (project licenses ~$99-319)	Full DIT station: offload+verify+metadata/lens data+look management (CDL/LUT), and (Lab only) audio sync + transcode/dailies — a much bigger tool than this event likely needs unless additional standalone cameras with real production metadata needs are added
FoolCat	Hedge	$29/mo, $89-129/activation, $299 bundle	Companion **report generator**, not a copy tool — HTML/PDF camera reports with thumbnails, plugs directly into OffShoot's offload queue

Tool	Vendor	Price	Role
o/PARASHOOT	OTTOMATIC GmbH	Free	Companion **card-safety** tool — confirms a card's files exist at the backup destination (filename+size, not checksum) then reversibly blanks the card so the camera prompts a clean reformat; pairs with OffShoot

Recommendation for this event's actual scale: ShotPut Pro or OffShoot Pro as the primary checksummed-copy tool — both are cheap, reliable, and exactly matched to "offload a drive/card, verify it, get a report" without paying for Silverstack's much larger metadata/look-management scope this design doesn't currently need (no standalone camera color pipeline in scope, no lens metadata being tracked). Add **FoolCat** only if stakeholders want visual per-session camera reports (director/producer-facing dailies-style PDFs) rather than just a copy-verification log. Add **o/PARASHOOT** (it's free) if any actual camera SD/CFexpress cards are in play and get reformatted/reused between sessions — cheap insurance against a card getting wiped before its footage is confirmed safe. **Silverstack XT/Lab is the right call instead** only if this event's scope grows to include standalone cameras needing real lens/metadata tracking or an in-house look/color pipeline — worth revisiting if that changes, not a default recommendation for the design as currently scoped.

A typical chained workflow, once decided: **ShotPut Pro/OffShoot (checksummed copy to the edit-suite NAS) → FoolCat (camera report, optional) → o/PARASHOOT (safe card reformat, optional)** — not competing alternatives, a real multi-stage pipeline other productions run this way.

Devices — new addition to the inventory

Device	Proposed IP	Notes
Edit suite NAS (Cloud Store or generic 10GbE NAS)	`192.168.22.2`	See Decision 1
MacBook Pro — Editor 1	`192.168.22.11`	10GbE via Thunderbolt adapter/dock
MacBook Pro — Editor 2	`192.168.22.12`	10GbE via Thunderbolt adapter/dock
Mac mini — Project Server + Remote Render	`192.168.22.20`	Dedicated always-on machine, not an editor's own laptop — see Decision 2

See `docs/ip-address-map.md` for how this fits the rest of the network's addressing.

Open questions this section introduces

- **How much of the event's footage actually needs editing access** — the full 8TB recording pool, or a curated/selected subset? Drives NAS capacity sizing in Decision 1.
- **Whether combining Project Server and Remote Render duty on one Mac mini holds up in practice** — Decision 2 flags this as architecturally sound but thinly documented by Blackmagic and practitioners alike; worth a real test before the event, not just before this doc.
- **Whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the physical-media DIT workflow has real cards to ingest beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data features become worth their cost.

- **Whether the Cloud Store Mini's RAID 0 (no redundancy) is an acceptable risk** given the fallback is a fresh pull from the mothership's recording pool, not permanent data loss — but still a real mid-event disruption if it happens.
- **Implementing the dual-write in the actual ingest scripts** — `pull-iso.py` and `mount-and-sync.sh` currently write only the Nextcloud destination; the second write this whole section depends on is a real code change, tracked as `docs/open-questions.md` item 15.